Assignment 3

Multi-Agent Game Playing in Stochastic Environments

Instructions

- This is an individual assignment. Demos will be conducted shortly after the submission date. Missing a demo results in zero marks.
- All parties involved in any form of plagiarism or cheating will receive zero marks across all assignments.
- Assignment deadline will not be extended. Late submissions will not be accepted.
- Submission guidelines must be followed exactly, otherwise your submission will not be accepted:

1. Create a new directory.
2. Rename it: YOURSECTION_ROLLNUMBER_NAME (e.g., A_20I-0012_Muhammad Ahmad).
3. Place all your files inside this directory.
4. Compress to a .zip file and submit on Google Classroom.

- Understanding the problem is part of the assignment. Read the entire document carefully before writing any code.
- The GUI must be implemented using Dear PyGui (DPG) exclusively. No other GUI library is permitted.
- Your solution must be generalised; it must work correctly on any valid board configuration file, not just the provided example.
- Marks for all sections are subject to change during the demo based on the examiner's understanding of your code and runtime behaviour.

1. Objective

The objective of this assignment is to implement and analyse game-playing agents in a stochastic, multi-agent environment using adversarial search. You will design a Stochastic Battlefield Game where three AI agents compete for territory, each with a different cognitive capability modelled through search-depth limits and observable range. You must implement Expectiminimax search with Alpha-Beta pruning, a custom evaluation function, a per-move node reporting mechanism, and a live visual interface built with Dear PyGui.

2. Problem Overview

Adversarial agents in the real world rarely operate with perfect information or equal capability. This assignment simulates that dynamic. Three agents -- Expert, Intermediate, and Novice -- compete on a 2D grid battlefield. Between agent turns, probabilistic Chance Nodes model real-world uncertainty through a 9-sided die with uneven probabilities. Each agent must plan strategically within its own cognitive limits, making the evaluation function essential when the search horizon is reached.

3. The Stochastic Battlefield -- Game Rules

3.1 Board Setup

The game is played on a configurable N x M grid (recommended: 8x8 for testing, 10x10 for final evaluation). Each cell is one of the following:

Cell Type	Description
Empty (.)	Neutral; can be captured by any agent.
Owned (A/B/C)	Controlled by the indicated agent. Contributes +1 score per owned cell per round.
Fortress (F)	Provides +3 score per round instead of +1. Has a defense value of 2 (see Section 3.3). Requires the defense value to reach 0 before it can be captured.
Obstacle (X)	Impassable. No agent or chance event affects this cell.
Minefield (M)	Traversable but triggers a Minefield Chance Event when any agent moves onto it (see Section 3.4.3).

3.2 Agents

There are exactly 3 agents: Agent A (Expert), Agent B (Intermediate), and Agent C (Novice). Their properties are fixed as follows:

Agent	Label	Max Depth	Starting Position	Capability Notes
Expert	A	7	Top-left corner	Full board visibility; all chance branches; transposition table.
Intermediate	B	5	Top-right corner	5-cell radius visibility; all chance branches; no transposition table.
Novice	C	3	Bottom-center	3-cell radius visibility; top-2 most probable chance branches only; greedy eval.

3.3 Actions and Agent Stats

3.3.1 Energy

Each agent starts with an energy pool of 20 units. Every action (Move, Attack, Fortify, or Wait) costs 1 energy. Energy does not regenerate during the game. When an agent's energy reaches 0, it may only Wait for the remainder of the game. Energy is tracked per agent (not per unit); both units of the same agent share the same energy pool.

3.3.2 Defense Value

Every owned cell has a defense value, which represents how difficult it is to capture. Defense values work as follows:

- A regular owned cell starts with a defense value of 1.
- A Fortress cell starts with a defense value of 2.
- When an attack on a cell results in a Full Success or Critical Hit (see Section 3.4.1), the cell's defense value is reduced by 1. The cell is only captured when its defense value reaches 0.
- When an attack results in a Partial Success, the defense value is not reduced; instead the cell becomes neutral (Empty) regardless of its current defense value.
- The Fortify action increases the defense value of one currently owned cell by 1, up to a maximum of 3. This costs 1 energy and takes the agent's entire turn for that unit.

Example: A Fortress cell (defense value 2) that has been successfully attacked once has defense value 1. A second Full Success attack on the same cell reduces it to 0 and the attacker captures it.

3.3.3 Actions

On each turn, an agent selects one action for each of its units (each agent starts with 2 units):

- Move: Move a unit to an adjacent cell (4-directional). Moving onto an empty cell captures it immediately. Moving onto an opponent-owned cell triggers an attack (resolved by the 9-sided die). Costs 1 energy.
- Attack: Attack an adjacent opponent-owned cell without moving onto it. Outcome is resolved by the 9-sided die (see Section 3.4.1). Costs 1 energy.
- Fortify: Increase the defense value of one currently owned adjacent cell by 1 (max 3). Costs 1 energy. The unit does not move.
- Wait: Do nothing. Costs 1 energy.

3.4 Stochastic Events -- Chance Nodes

After every agent turn, a Chance Node is evaluated. This is what distinguishes this game from a deterministic setting.

3.4.1 9-Sided Die for Combat (Uneven Probabilities)

When an agent attacks or moves onto an opponent cell, a virtual 9-sided die is rolled. The sides are assigned uneven probabilities as follows:

Die Faces	Probability	Outcome	Effect on Target Cell
1, 2	0.10 each (0.20 total)	Attack fails	Defender retains cell; no change to defense value. Attacker loses 1 additional energy.

3	0.15	Attack fails	Defender retains cell; no change to defense value. No additional energy loss.
4, 5	0.08 each (0.16 total)	Partial success	Cell becomes neutral (Empty) regardless of defense value. Defense value resets.
6	0.12	Partial success	Cell becomes neutral (Empty); attacker's unit advances onto the now-empty cell.
7, 8	0.13 each (0.26 total)	Full success	Defense value of target cell is reduced by 1. If defense value reaches 0, attacker captures the cell.
9	0.11	Critical hit	Defense value reduced by 1. If defense value reaches 0, attacker captures cell AND gains +2 bonus score.

Verify: 0.10 + 0.10 + 0.15 + 0.08 + 0.08 + 0.12 + 0.13 + 0.13 + 0.11 = 1.00

3.4.2 Environmental Events

At the start of each round (before Agent A moves), one environmental event is drawn uniformly at random (probability 0.25 each):

- Supply Drop: A randomly selected empty cell becomes a Fortress (defense value 2) for 3 rounds, after which it reverts to Empty.
- Earthquake: One randomly selected owned cell (any agent) has its defense value reduced by 1. If the defense value reaches 0, the cell reverts to Empty.
- Reinforcement: The agent with the lowest current score gains one extra unit for 2 turns. The extra unit is placed on any empty cell adjacent to that agent's existing unit (or starting position if no unit is on the board). If no adjacent empty cell exists, the extra unit is placed on the agent's nearest owned cell.
- Fog of War: For this turn only, each agent's observable range is halved (rounded down). Agents cannot search cells outside their range.

3.4.3 Minefield Chance Event

When any agent's unit moves onto a Minefield (M) cell, the following 4-sided event is triggered immediately, before the agent's turn ends:

Outcome	Probability	Effect
Safe passage	0.40	No effect. Unit continues normally.
Energy drain	0.30	The moving unit's agent loses 3 energy immediately.
Unit disabled	0.20	The unit that triggered the mine cannot act for the next 2 turns.
Mine detonation	0.10	The Minefield cell becomes an Obstacle (X) permanently. The moving unit's agent loses 5 energy.

The Minefield cell itself is not consumed unless a Mine Detonation occurs. Multiple units may trigger the same Minefield on separate turns.

3.5 Game Termination

The game ends when one of the following conditions is met:

- A fixed number of rounds is reached (configurable; default R = 30 rounds).
- An agent controls more than 60% of all non-obstacle cells -- that agent wins immediately.
- An agent loses all units or runs out of energy -- that agent is eliminated. The game continues among remaining agents.

3.6 Scoring

- +1 point per owned regular cell per round.
- +3 points per owned Fortress cell per round.

- +5 bonus points for eliminating an opponent.

The agent with the highest cumulative score at game end is the winner.

4. Required Algorithm: Expectiminimax with Alpha-Beta Pruning

4.1 Overview

You must implement the Expectiminimax algorithm as the core search backbone for all three agents. Expectiminimax extends Minimax by introducing Chance Nodes between MAX and MIN nodes. The game tree alternates between:

- MAX Node: The current agent's turn. Choose the action maximising expected utility.
- MIN Node: Opponent agents' turns. Opponents are assumed to act optimally to minimise your score.
- CHANCE Node: Stochastic die outcomes. The value is the probability-weighted average of child node values, using the uneven 9-sided probabilities defined in Section 3.4.1.

4.2 Tree Structure

A single ply in the game tree is structured as follows:

```
Agent A turn (MAX, depth = 7)
  └── Chance Node  (9-sided die; uneven probabilities)
       ├── [p=0.20] fail outcomes     →  Agent B turn (MIN, depth = 5)
       │                                     └── Chance Node
       │                                          └── Agent C turn (MIN, depth = 3)
       ├── [p=0.31] partial outcomes  →  ...
       └── [p=0.49] full/critical     →  ...
```

4.3 Alpha-Beta Pruning

Alpha-Beta pruning must be applied on MAX and MIN nodes. Pruning is NOT applied at Chance Nodes -- all branches must be evaluated for correct probability weighting.

For each move made during the game, your program must print a per-move node report (see Section 6.2).

4.4 Depth Limiting and Evaluation Function

Each agent cuts off its search at its personal depth limit. At cutoff, the Evaluation Function estimates the utility of the board state. You must design your own Evaluation Function incorporating at least the following factors:

- Score differential: Difference between this agent's score and the average opponent score.
- Territory control: Number of owned cells, weighted Fortress count.
- Positional advantage: Proximity of units to high-value cells and Fortress tiles.
- Threat assessment: Number of opponent units adjacent to owned cells (defensive penalty).
- Energy advantage: This agent's remaining energy minus the average opponent energy remaining.

Your report must justify every term in your evaluation function with a clear argument for why it correlates with long-term winning. The Novice agent uses a greedy evaluation (score differential only). The Expert uses the full function.

5. Agent Capability Asymmetry

The three agents must NOT have equal capability. This asymmetry must be enforced structurally in code, not as surface-level if-else branching on agent ID.

Capability	Expert (A)	Intermediate (B)	Novice (C)
Search Depth	7	5	3
Observable Range	Full board	5-cell radius	3-cell radius
Chance Branches Evaluated	All 9 faces	All 9 faces	Top-2 most probable faces only
Evaluation Function	Full (5 factors)	3 factors	Greedy (score diff only)
Transposition Table	Yes	No	No

6. Input, Output, and GUI

6.1 Input File: board.txt

Your program reads the initial board configuration from board.txt in the following format:

```
N M R
<N lines of M characters each, from top row to bottom row>
A_startX A_startY
B_startX B_startY
C_startX C_startY
```

Each grid character must be one of: . (empty), X (obstacle), F (fortress), M (minefield). Your solution must be fully generalised and must work correctly on any valid board.txt, including different grid sizes, different obstacle layouts, and different starting positions.

6.2 Per-Move Node Report (Console + results.txt)

After every move by any agent, your program must print the following to both the console and results.txt:

```
Move 12 | Agent A (Expert) | Action: Attack (3,4)
  Expectiminimax nodes explored : 4,821
  Nodes pruned (Alpha-Beta)     : 1,203  (24.9%)
  Chosen action value (utility) : 0.73
```

At the end of the game, print a summary table with total nodes explored, total pruned, and pruning efficiency (%) for each agent across all moves.

6.3 GUI Requirements (Dear PyGui only)

You must build a visual interface for the game using Dear PyGui (DPG). No other GUI library is permitted. The GUI must show:

- The game board rendered as a grid, with distinct colours for each agent's owned cells, Fortress tiles, Obstacles, and Minefields.
- Each agent's current score, number of units, remaining energy, and owned cell count displayed in a side panel.
- A move log panel showing each agent's last action and the chance event outcome for that turn.
- Step-through controls: Next Move button to advance one move at a time, and a Run button to play the game at a configurable speed.
- Per-move node statistics (nodes explored / pruned) displayed live after each move.

6.4 Output File: results.txt

results.txt must contain the per-move node reports (Section 6.2), the final scores, and the winner declaration.

7. Generalisation Requirement

Your implementation must be fully generalised. This means:

- Board dimensions N and M are read from board.txt and must not be hardcoded.
- Number of rounds R is read from board.txt and must not be hardcoded.
- Agent starting positions are read from board.txt.
- The 9-sided die probability table and the Minefield event probability table must each be stored as a configurable data structure (e.g., a dictionary or list), not hardcoded inline throughout the search logic.
- The evaluator and search logic must work correctly regardless of grid size, number of Fortress tiles, obstacle layout, or starting positions.

During the demo, the examiner may substitute a different board.txt to verify generalisation. Hardcoded solutions will receive zero marks for this section.

8. Statistical Analysis

You must include a quantitative analysis in your report comparing the three agents across at least 5 independent game runs on the same board configuration, reporting:

- Win rate of each agent.

- Average final score per agent.
- Average nodes explored per agent per turn, and average pruning efficiency.
- Impact of depth limit: Run Agent A at depth 5 and 7 and compare outcomes across 5 runs.
- Impact of stochasticity: Run 5 games with chance events disabled (deterministic mode) vs. 5 with chance events enabled; discuss the difference in agent rankings.

9. Example Board Configuration

The following 8x8 board is provided for initial testing only. Your solution must work on any valid board:

```
8 8 30
........
.X..F...
....X...
..M.....
.......M
...X....
..F..X..
........
0 0
7 0
3 7
```

Agent A starts at (0,0) top-left, Agent B at (7,0) top-right, Agent C at (3,7) bottom-center.

10. Marks Breakdown

All marks are subject to revision during the demo based on the examiner's assessment of your understanding and the runtime behaviour of your code.

Component	Marks	Demo-Adjustable
Correct Expectiminimax (MAX, MIN, and CHANCE nodes with 9-sided uneven die)	20	Yes
Alpha-Beta pruning on MAX/MIN nodes (not Chance Nodes)	10	Yes
Agent capability asymmetry (depth, range, branches, eval complexity)	15	Yes
Evaluation function with justified factors (Expert: 5, Intermediate: 3, Novice: greedy)	15	Yes
Per-move node report (explored / pruned per move, summary table)	10	Yes
Generalised solution (works on any valid board.txt)	10	Yes
Dear PyGui GUI (board render, agent panel, move log, step controls, live stats)	15	Yes
Report (algorithm design, eval justification, statistical analysis)	5	No
Total	**100**	

11. What to Submit

- assignment3.py -- a single Python 3 file that reads from board.txt, launches the Dear PyGui interface, and writes to results.txt. No external libraries beyond the Python standard library, Dear PyGui, and (optionally) matplotlib for report plots.
- board.txt -- the board configuration file used for your submission demo.
- results.txt -- the output generated by your program on your board.txt.
- A Word document report containing:
  1. State space representation for the Expectiminimax tree.
  2. Full Evaluation Function definition with justification for every term.

3. Depth-limiting and capability asymmetry design decisions.
4. Alpha-Beta pruning analysis: node count comparison with and without pruning.
5. Statistical analysis across 5 runs including stochasticity impact discussion.

End of Assignment 3